Practical No: 09

Aim:

To implement join-like operations using $lookup in MongoDB.

Theory:

MongoDB is a NoSQL document database and does not use traditional relational tables. However, real-world applications often require combining information stored across different collections. To achieve this, MongoDB provides the **$lookup** aggregation stage, which functions similarly to SQL joins by linking documents from two collections based on matching fields.

The $lookup stage allows users to perform a *left outer join* between the source collection (e.g., students) and the target collection (e.g., departments). It takes multiple parameters:

- **from** – the name of the foreign collection to join with
- **localField** – the field in the current collection to match
- **foreignField** – the field in the foreign collection to compare
- **as** – the name of the output array field that will store matched documents

In this experiment, the students collection contains student data, while the newly created departments collection stores details such as the department name and the head of department (HOD). By applying $lookup, MongoDB links each student's dept field with the corresponding dept field in the departments collection. As a result, each student's document will contain a new array field—deptInfo—which holds department details such as the HOD.

Setup:

```
db.departments.insertMany([ { dept: "CSE", hod: "Dr. Rao" }, { dept: "ECE", hod: "Dr. Devi" }
])
```

Commands:

```
db.students.aggregate([ { $lookup: { from: "departments", localField: "dept", foreignField:
"dept", as: "deptInfo" } } ])
```

OR

Students Collection

```
db.students.insertMany([
{ "_id": 1, "name": "Rahul", "course_id": 201 },
{ "_id": 2, "name": "Sneha", "course_id": 202 },
{ "_id": 3, "name": "Amit", "course_id": 201 }
])
```

Courses Collection

```
db.courses.insertMany([
{ "course_id": 201, "course_name": "Computer Science" },
{ "course_id": 202, "course_name": "Information Technology" }
])
```

`$lookup` Command

```
db.students.aggregate([
{
 $lookup: {
   from: "courses",
   localField: "course_id",
   foreignField: "course_id",
   as: "course_details"
 }
}
])
```

OUTPUT:


```
{

 "name": "Rahul",

 "course_id": 201,

 "course_details": [

   { "course_id": 201, "course_name": "Computer Science" }

 ]

}
```


Viva Questions:

1. What is $lookup in MongoDB?

$lookup is an aggregation stage used to combine documents from two collections based on matching fields, similar to SQL JOIN.

2. Which type of join does $lookup perform?

$lookup performs a **left outer join**.

3. What does the from field specify in $lookup?

It specifies the foreign collection from which documents are joined.

4. What is the purpose of localField?

localField specifies the field in the source collection used for matching.

Conclusion:

In this experiment, join-like operations were successfully implemented using the $lookup stage of the MongoDB aggregation pipeline. Student records from the students collection were combined with department details from the departments collection based on a common field. The results demonstrate how MongoDB supports relational-style data retrieval without using traditional SQL joins, making it suitable for flexible and scalable applications.